

Arrays

Bib: Java: An Introduction to Problem Solving & Programming, Walter Savitch

Cap. 7: Arrays

Programação III (PG3)

João Pedro Patriarca, (joao.patriarca@isel.pt),

Gabinete F.0.23 do edifício F, LEETC, ADEETC



Resumo

- *Array* - fundamentos
- Ordenação e pesquisa em *arrays*
- *Array* multidimensional

Resumo

- *Array* - fundamentos
- Ordenação e pesquisa em *arrays*
- *Array* multidimensional

Array – o que é?

- Por definição, corresponde a uma sequência de elementos em memória de um determinado tipo
 - Podem ser tipos primitivos correspondendo a uma sequência de valores do tipo especificado
 - Podem ser tipos classe correspondendo a uma sequência de referências para objetos em memória
 - Os objetos não têm de estar em sequência em memória mas as referências estão
- Equivalente a um tipo classe com notação própria
 - Na linguagem Java não existe uma classe que represente um *array*
- Regra geral, os nomes dos *arrays* são atribuídos no singular

Criação de um *array* (1 de 2)

- Possível criar *arrays* através de diferentes formas
 - Forma 1: declaração apenas; criação posterior
`<type>[] <name>; ...; <name> = new <type>[<length>];`
Ex: `double[] a; ...; a = new double[100];`
 - Forma 2: declaração com criação
`<type>[] <name> = new <type>[<length>];`
Ex: `String[] b = new String[10];`
 - Forma 3: declaração apenas; criação e iniciação posterior
`<type>[] <name>; ...; <name> = new <type>{ <elem1>, ..., <elemN> };`
Ex: `long[] l; ...; l = new long[]{10, 20, 30, 40};`
 - Forma 4: declaração, criação e iniciação, em simultâneo
`<type>[] <name> = { <elem1>, ..., <elemN> };`
Ex: `char[] c = {'a', 'b', 'c', 'd'};`

Criação de um *array* (2 de 2)

- Forma 1: `<type>[] <name>; ...; <name> = new <type>[<length>];`
- Forma 2: `<type>[] <name> = new <type>[<length>];`
- Forma 3: `<type>[] <name>; ...; <name> = new <type>{ <elem1>, ..., <elemN> };`
- Forma 4: `<type>[] <name> = { <elem1>, ..., <elemN> };`
- Na forma 1 e 2, no processo de criação, as entradas são iniciadas com valores por omissão, função do tipo de elementos
 - Com 0, se tipos inteiros e reais; com false, se tipo boolean; com null, se tipo classe (não são criados os objetos)
- Na forma 3 e 4, a dimensão do *array* é definida com base no número de elementos especificados na iniciação
- A dimensão é determinada na criação e é imutável durante o tempo de vida do *array*
- Um objeto do tipo *array* dispõe da variável de instância pública `length` que devolve a capacidade do *array*

Acesso a um *array*

- Parênteses retos e um índice para aceder a um elemento

```
a[idx] = value; // altera o valor no array a, no índice idx, com value
int v = a[idx]; // armazena em v o valor do array a, no índice idx
```
- O índice pode ser qualquer expressão que produza um valor inteiro
- O índice 0 acede ao primeiro elemento
- O índice `length-1` acede ao último elemento
- A instrução *for-each* itera sobre todos os elementos
- Gera a exceção `IndexOutOfBoundsException` na utilização de um índice inválido (menor que 0 ou maior ou igual a `length`)

Arrays em classes e métodos (1 de 3)

- *Arrays* podem constituir variáveis de instância ou estáticas de uma classe

```
public class X { private int[] field1;  
                private static double[] field2; ... }
```

- O parâmetro de um método pode ser uma variável indexada
 - O tipo do parâmetro depende do tipo de elementos do *array*
 - Para tipos primitivos, o valor é passado por cópia
 - Para tipos classe, a referência para o objeto é passada por cópia, logo, é possível alterar o estado do objeto (assim a classe do elemento o permita)

```
public static int doSomething(double v, String s) { ... }
```

- Chamada:

```
double[] a = { ... }; String[] b = new String[ARRAY_SIZE]; ...;  
X.doSomething(a[a.length-1], b[0]);
```

Arrays em classes e métodos (2 de 3)

- O parâmetro de um método pode ser um *array*
 - Permite alterar os valores do *array*; não permite alterar a referência do *array*

```
public static void incByN (int[] a, int n) { ... }
```

- Chamada:

```
int[] c = new int[ARRAY_SIZE]; ...; Y.incByN(c, N);
```

- Método `main`

```
public static void main(String[] args) { ... }
```

- O parâmetro `args` do método `main` representa os argumentos que são passados à aplicação, aquando o momento de execução:

```
> java Xpto <arg1> <arg2> <arg3>
```

- `args[0]` representa o primeiro argumento (`<arg1>`)
- Para obter o nome da aplicação: `Xpto.getClass().getName()`

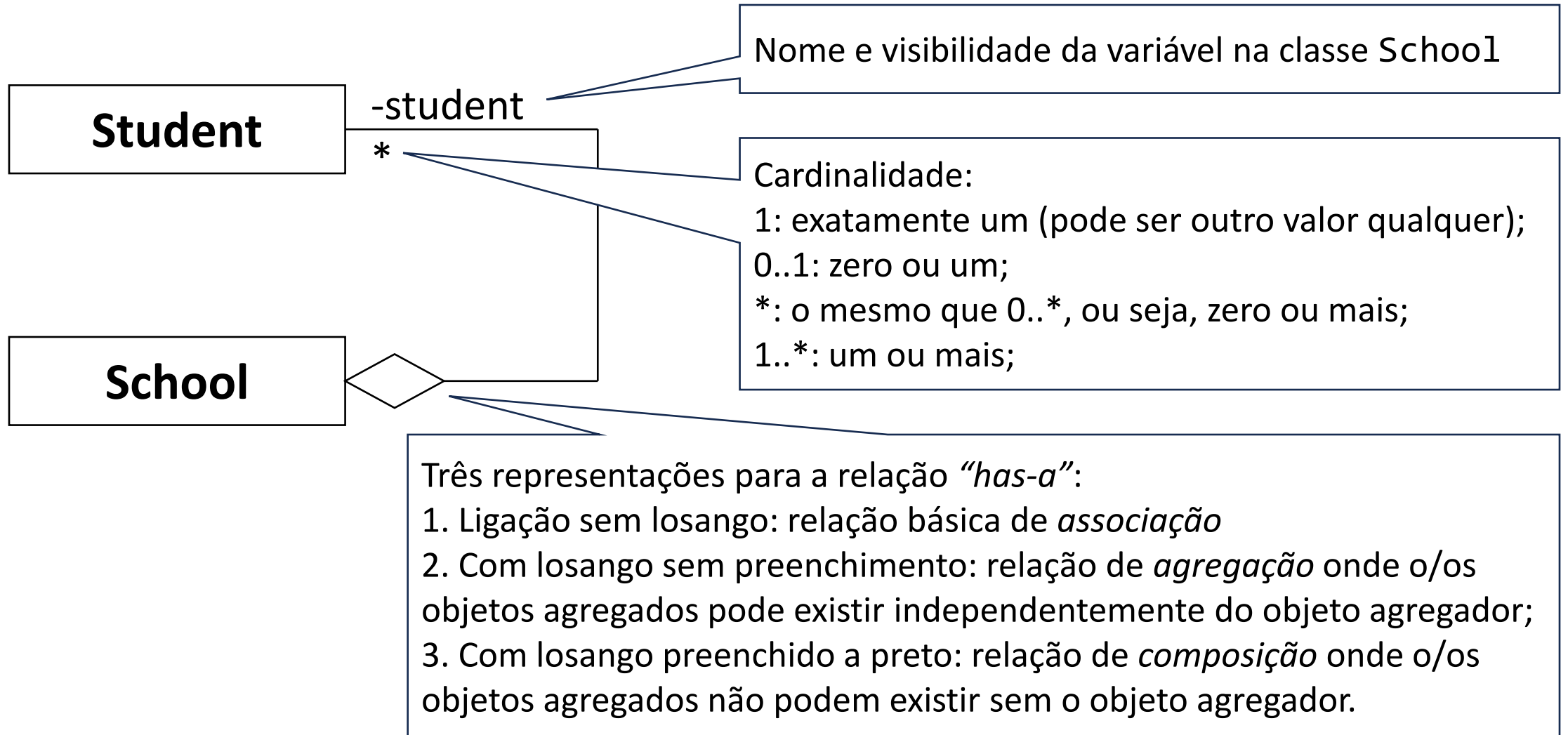
Arrays em classes e métodos (3 de 3)

- *Array* como retorno de um método
 - Tal como para um objeto de outra classe, retorna uma referência para o objeto *array*

```
public class W {  
    private static char[] alpha = {'a', 'b', ..., 'z'};  
    public static char[] getAlphabet() {  
        return alpha;  
    }  
}
```

- Sofre da mesma preocupação “*privacy leak*”, apresentado no tópico “Definição de classes”

Relação “*has-a*” em UML



Operador = e ==

- Como *arrays* são objetos do tipo classe, o comportamento dos operadores = e == é o mesmo que no âmbito de objetos de outras classes
 - Operador = coloca duas variáveis a referenciar o mesmo objeto *array*
`int[] a = { ... }; int[] b = a;`
 - Operador == testa se duas referências são iguais; não testa se os elementos nos *arrays* são iguais
`boolean c = a == b;`

Arrays parcialmente preenchidos

- Um *array* não tem de estar completamente preenchido
 - É da responsabilidade do autor do programa manter estado de quantos elementos válidos permanecem no *array* e como estão arrumados no *array*
- Tipicamente, “parcialmente preenchido”, significa preenchido de 0 a N e sem valores significativos de N+1 a length-1
- É necessário manter em memória uma variável (ex: `size`) com o número de elementos válidos no *array*
- Num preenchimento não linear (de 0 a N), tipicamente usam-se valores inválidos no domínio da aplicação para indicar valores sem significado no *array*
 - Para tipos classe é usado tipicamente o valor `null`

Método com número de argumentos variável (*Varargs*)

- Funcionalidade adicionada na versão JavaSE 5
- Documentation:
<https://docs.oracle.com/javase/8/docs/technotes/guides/language/varargs.html>
- Permite a especificação de um número de argumentos variável na chamada ao método

```
System.out.printf("Ex1: %d \"%s\" %.2f\n", 10, "xpto", 5.7);  
System.out.printf("Ex2: \'%c\' 0x%x\n", 'Z', 0xABCD);
```

- Tem de ser obrigatoriamente o último parâmetro do método
`public PrintStream printf(String format, Object... args);`
- Internamente é implementado com um *array*
- Permite a especificação de um tipo qualquer (primitivos e classes)

Varargs - exemplo

```
public class VarargsDemo {  
    public static void demo(String title, String prefix, Object... arg) {  
        System.out.println(title);  
        for(Object o: arg) {  
            System.out.println(prefix + o);  
        }  
        System.out.println(arg[0]);  
        System.out.println(arg[arg.length-1]);  
    }  
  
    public static void main(String[] args) {  
        demo("Demo1", "\\t- ", (int)'0', '0', 1, true, "xpto", 10.7);  
    }  
}
```

Declaração. Permite a especificação de um tipo qualquer

Internamente é implementado com um *array*

Argumentos

Demo1

- 48
- 0
- 1
- true
- xpto
- 10.7

48
10.7

Resumo

- *Array* - fundamentos
- Ordenação e pesquisa em *arrays*
- *Array* multidimensional

Ordenação

- Método `sort` do tipo `java.util.Arrays`
 - Ordena um *array* com qualquer tipo de elementos (primitivos e classes)
 - O método `sort` está sobrecarregado
- A ordenação de *arrays* de tipos primitivos é realizada por ordem crescente, segundo a ordem natural dos elementos
- A ordenação de tipos compostos implica que os objetos sejam comparáveis
 - Para um objeto ser comparável tem de implementar a interface `Comparable` que define o método `compareTo`
 - O método `compareTo` retorna um inteiro indicador do resultado da comparação
 - A ordenação implementada por `Comparable` refere-se como a *ordem natural* da classe
- Fortemente recomendável que a ordem natural seja *consistente* com o método `equals`. Esta consistência é obtida se para todos os objetos da classe
 - O método `e1.compareTo(e2)` retornar 0 e `e1.equals(e2)` retornar `true`
 - O método `e1.compareTo(e2)` retornar diferente de 0 e `e1.equals(e2)` retornar `false`
 - Exceção para `e2 = null`: no método `compareTo` gera exceção; no método `equals` retorna `false`

Interface Comparable<T>

- Interface usada para impor uma ordem nos objetos que a implementam
- Pertence à *framework* de coleções Java (*package java.util*)
- Usada para ordenar elementos dentro de uma coleção
- Declara um único método (denominada, por isso, por interface funcional)
`public int compareTo(T other);`
 - Retorna um inteiro negativo, zero, ou um inteiro positivo função do objeto sobre o qual o método é chamado for inferior, igual ou superior ao objeto especificado por `other`
 - Tipo T: traduzido por um tipo classe quando uma classe implementa esta interface
`public class Xpto implements Comparable<Xpto> { ... }`
- Impõe algumas regras de semântica:
 - Relação transitiva: se $A < B$ e $B < C$ então $A < C$

Método `java.util.Arrays.sort`

```
static void sort ( Object[] a )  
static void sort ( Object[] a, int fromIndex, int toIndex );
```

- O primeiro método ordena os objetos do *array* na totalidade
- O segundo método ordena os objetos do *array* entre os índices `fromIndex` e `toIndex`
- O algoritmo de ordenação é estável (elementos iguais não são reordenados) e garante complexidade $O(n \log n)$

Exemplo Arrays.sort

```
public static boolean hasSameStrings(String[] a1, String[] a2) {  
    if ( a1 == a2 ) return true;  
    if ( a1 == null || a2 == null ) return false;  
    if ( a1.length != a2.length ) return false;  
    Arrays.sort( a1 ); // Ordenar a1  
    Arrays.sort( a2 ); // Ordenar a2  
    // Verificar se são iguais  
    for ( int i= 0; i < a1.length; ++i )  
        if ( !a1[i].equals( a2[i] ) ) return false;  
    return true;  
}
```

Pesquisa sobre um *array* desordenado

- Um *array* desordenado exige uma pesquisa sequencial
 - Implica uma iteração de 0 a $\text{length}-1$ para determinar se o elemento a pesquisar está presente no *array*
 - Interrompe a iteração se o elemento é encontrado
- Considerando o exemplo de um *array* com 10 inteiros

10	4	7	6	9	1	5	3	2	8
0	1	2	3	4	5	6	7	8	9

- A pesquisa pelo valor 11 implica a iteração sobre todos os elementos para concluir que não existe no *array*
- A pesquisa pelo valor 4 envolve 2 iterações

Pesquisa sobre um *array* ordenado

- Um *array* ordenado permite efetuar pesquisas mais eficientes
 - Pesquisa binária ou dicotômica
 - Seleciona o elemento do meio do *array* eliminando da pesquisa metade dos elementos
 - Este critério é aplicado sucessivamente até encontrar o elemento ou determinar que o elemento não está presente no *array*

- Considerando o exemplo:

1	2	3	4	5	6	7	8	9	10
0	1	2	3	4	5	6	7	8	9

- A pesquisa pelo valor 11 envolve 3 iterações para concluir que não existe no *array*
- A pesquisa pelo valor 6 envolve 1 iteração
- A pesquisa pelo valor 4 envolve 4 iterações

Exemplo da pesquisa pelo valor 11

- Considera-se a pesquisa no intervalo determinado por `leftIdx` (menor índice, inclusive), `rightIdx` (maior índice, exclusive) e o índice do meio calculado através da expressão
$$\text{leftIdx} + (\text{rightIdx} - \text{leftIdx}) / 2$$
- As condições de paragem são: elemento encontrado no índice `midIdx` ou `leftIdx >= rightIdx`

1. `leftIdx = 0; rightIdx = 10; midIdx = 0+(10-0)/2 = 5`

- No índice 5 está o valor 6, logo metade superior (6..10)

1	2	3	4	5	6	7	8	9	10
0	1	2	3	4	5	6	7	8	9

2. `leftIdx = 6; rightIdx = 10; midIdx = 6+(10-6)/2 = 8`

- No índice 8 está o valor 9, logo metade superior (9..10)

7	8	9	10
6	7	8	9

3. `leftIdx = 9; rightIdx = 10; midIdx = 9+(10-9)/2 = 9`

- No índice 9 está o valor 10, logo metade superior (10..10)

10
9

- Condição de paragem atingida: `leftIdx >= rightIdx`

Exemplo da pesquisa pelo valor 4

1. $\text{leftIdx} = 0$; $\text{rightIdx} = 10$; $\text{midIdx} = 0 + (10 - 0) / 2 = 5$

- No índice 5 está o valor 6, logo metade inferior (0..5)

1	2	3	4	5	6	7	8	9	10
0	1	2	3	4	5	6	7	8	9

2. $\text{leftIdx} = 0$; $\text{rightIdx} = 5$; $\text{midIdx} = 0 + (5 - 0) / 2 = 2$

- No índice 2 está o valor 3, logo metade superior (3..5)

1	2	3	4	5
0	1	2	3	4

3. $\text{leftIdx} = 3$; $\text{rightIdx} = 5$; $\text{midIdx} = 3 + (5 - 3) / 2 = 4$

- No índice 4 está o valor 5, logo metade inferior (3..4)

4	5
3	4

4. $\text{leftIdx} = 3$; $\text{rightIdx} = 4$; $\text{midIdx} = 3 + (4 - 3) / 2 = 3$

- Encontra o valor 4 no índice 3; condição de paragem atingida

4
3

Método `java.util.BinarySearch`

```
static int binarySearch ( Object[] a, Object key )  
static int binarySearch ( Object[] a, int fromIndex,  
                          int toIndex, Object key );
```

- Os objetos devem estar ordenados por ordem crescente de acordo com a sua ordem natural
- Retorna o índice onde encontra o objeto `key`
- Retorna $-(\text{ponto de inserção}) - 1$, caso não encontre o objeto `key`
 - Ponto de inserção:
 - Índice onde o objeto `key` pode ser inserido sem alterar a ordem dos objetos
 - Corresponde ao índice do primeiro objeto maior que o objeto `key` ou a `a.length` se o objeto `key` for maior que todos os objetos no *array*

Exemplo Arrays.binarySearch

```
public static boolean insertSorted( String[] sorted, int size, String s) {  
    int index = Arrays.binarySearch( sorted, 0, size, s );  
    if ( index >= 0 ) return false; // O elemento já existe  
    if ( size == sorted.length ) throw new ArrayStoreException("No capacity to new element");  
    int insertionPoint = -(index + 1);  
    System.arraycopy( sorted, insertionPoint, sorted, insertionPoint+1,  
        size - insertionPoint );  
    sorted[ insertionPoint ] = s;  
    return true;  
}
```

Resumo

- *Array* - fundamentos
- Ordenação e pesquisa em *arrays*
- *Array* multidimensional

Declaração e criação

- São usadas as mesmas regras que nos *arrays* unidimensionais

- Syntaxe

`<type>[][]...[] <name> = new <type>[length1][length2] ... [lengthN];`

- Exemplos

`char[][] page = new char[100][80]; // 100 linhas; 80 colunas`

`int[][] table = new int[10][6];`

`double[][][] picture3D = new double[10][20][30]; // 10 matrizes de 20 linhas por 30 colunas`

`AnyClass[][] anyClass = new AnyClass[30][20];`

- Para determinar o número de elementos de um *array* (table)

`int tableLines = table.length; // 10`

`int tableColumns = table[0].length; // 6`

`int tableElems = tableLines * tableColumns; // 60`

Ragged arrays (arrays irregulares)

- Internamente, um *array* bidimensional é um *array* unidimensional de *arrays* unidimensionais
 - O mesmo modelo se aplica a um *array* multidimensional com qualquer número de níveis

```
int[][] table = new int[2][3];
```
 - É equivalente a

```
int[][] table = new int[2][];  
table[0] = new int[3]; table[1] = new int[3];
```
- Os *arrays* podem ter dimensões diferentes (*ragged arrays*)

```
int[][] table = new int[2][];  
table[0] = new int[3]; table[1] = new int[5];
```